

Module 0 — Python Fundamentals

Core Python for AI development

What this module covers

This module covers the Python patterns used throughout the course.

Before we build a single agent, we need a solid foundation.

Data structures, modules, async, and HTTP — the core toolkit.

🎯 Learning goals

- Work fluently with **lists, dicts, sets, tuples**.
- Organise code with **modules**, CLI args, and **logging**.
- Model domain objects with **dataclasses** and **Protocol**.
- Write **async** code: tasks, queues, timeouts, cancellation.
- Build and test a basic **HTTP API** with FastAPI + httpx.

Data structures

- **Lists** — ordered, mutable, iterable. User records, event logs.
- **Dicts** — O(1) key-value lookup. Lookup tables, configuration.
- **Sets** — unique elements, fast membership, set algebra. Tags, categories.
- **Tuples** — immutable sequences. Fixed records, function returns.

List comprehensions

```
users = load_json("users.json")

# Filter + transform in one expression
active_engineers = [
    u["name"]
    for u in users
    if u["role"] == "engineer"
    and u["active_project"] is not None
]
```

- → Comprehensions replace verbose for-loops for filter+map
- → Readable, efficient, and Pythonic

Dict operations

```
# Build a lookup table
users_by_id = {u["id"]: u for u in users}
admin = users_by_id["USR-003"]

# Count by role
counts: dict[str, int] = {}
for u in users:
    role = u["role"]
    counts[role] = counts.get(role, 0) + 1
```

- Dict comprehensions for instant lookups
- .get(key, default) avoids KeyError



Demo — Data structures

Switch to terminal: python demo/01_data_structures.py



Modules, CLI + logging

Organising code for maintainable projects

Modules and packages

- `import json`, `from pathlib import Path` — standard library.
- `if __name__ == "__main__":` — script vs import guard.
- Packages = directories with `__init__.py` (or implicit namespace).
- Keep imports at the top; organise by stdlib → third-party → local.

CLI with argparse

```
import argparse

parser = argparse.ArgumentParser(
    description="User management CLI"
)
parser.add_argument("--department", "-d")
parser.add_argument("--min-clearance", "-c", type=int, default=0)
args = parser.parse_args()
```

- argparse is stdlib — no extra dependencies
- Type conversion, defaults, and help text built in

Logging — better than print()

- `logging.basicConfig(level=logging.INFO)` — one-line setup.
- Levels: DEBUG → INFO → WARNING → ERROR → CRITICAL.
- `logger.info("Loaded %d records", count)` — lazy formatting.
- Production: structured logs (JSON), not print statements.



Demo — Modules + CLI

Switch to terminal: python demo/02_modules_cli.py



Dataclasses + Protocol

Type-safe domain modelling without inheritance

Dataclasses and protocols

- `@dataclass` generates `__init__`, `__repr__`, `__eq__` for you.
- Type hints document the shape; defaults reduce boilerplate.
- `field(default_factory=list)` for mutable defaults.
- Immutable style: return new instances instead of mutating.

Dataclass example

```
from dataclasses import dataclass, field

@dataclass
class Employee:
    id: str
    name: str
    role: str
    access_level: int = 1
    skills: list[str] = field(default_factory=list)
    active_project: str | None = None

    @property
    def is_available(self) -> bool:
        return self.active_project is None
```

- → str | None — Python 3.10+ union syntax
- → Properties for derived state without extra storage

Protocol — contracts without inheritance

- `typing.Protocol` defines a structural interface.
- Any class with matching methods satisfies it — no base class needed.
- Perfect for agent components: tools, memory backends, formatters.
- Duck typing with type checker support.

Protocol in action

```
from typing import Protocol

class Briefable(Protocol):
    def briefing(self) -> str: ...

def print_briefings(items: list[Briefable]):
    for item in items:
        print(item.briefing())

# Order and Invoice both qualify —
# no shared base class needed
```

- → Structural subtyping: if it has .briefing(), it qualifies
- → Keeps agent code loosely coupled and testable



Demo — Dataclasses + Protocols

Switch to terminal: python demo/03_dataclasses_protocols.py



Async essentials

Concurrent I/O for agent workloads

Async essentials — why it matters for agents

- AI agents wait on network calls — LLM APIs, tool servers, databases.
- `async` / `await` lets one thread handle many concurrent waits.
- Key primitives: `asyncio.create_task`, `gather`, `Queue`, `wait_for`.
- `Task.cancel()` for cleanup when the user walks away.

Async producer / consumer

```
import asyncio

async def producer(queue: asyncio.Queue):
    for event in events:
        await queue.put(event)
    await queue.put(None)  # sentinel

async def consumer(queue: asyncio.Queue):
    while (item := await queue.get()) is not None:
        process(item)

queue = asyncio.Queue(maxsize=5)
await asyncio.gather(producer(queue), consumer(queue))
```

- → Queue with maxsize creates backpressure
- → None sentinel signals "no more data"

Timeouts and cancellation

```
# Timeout: abort if too slow
try:
    result = await asyncio.wait_for(
        long_scan(), timeout=2.0
    )
except asyncio.TimeoutError:
    print("Scan timed out")

# Cancellation: clean shutdown
task = asyncio.create_task(monitor())
await asyncio.sleep(5)
task.cancel()
```

- → wait_for wraps any coroutine with a deadline
- → CancelledError propagates for cleanup in try/except



Demo — Async essentials

Switch to terminal: python demo/04_async_essentials.py



HTTP basics

APIs and testing with FastAPI + httpx

HTTP basics — FastAPI + httpx

- **FastAPI** — modern, async, auto-generates OpenAPI docs.
- **httpx** — async-capable HTTP client (like requests but better).
- Path params, query params, JSON bodies — all typed.
- Test with `httpx.ASGITransport` — no real server needed.

FastAPI in 10 lines

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

@app.get("/projects")
async def list_projects(status: str | None = None):
    results = PROJECTS
    if status:
        results = [p for p in results if p["status"] == status]
    return {"count": len(results), "projects": results}
```

- → Type hints become query parameter validation
- → Return dicts/lists — FastAPI serializes to JSON



Demo — HTTP + FastAPI

Switch to terminal: python demo/05_http_basics.py



Putting it all together

Summary and exercises

Summary

Pick the right collection

Lists for order, dicts for lookup, sets for membership, tuples for fixed records.

Use dataclasses for domain objects

Dicts are fine for JSON; dataclasses are better for code.

async for I/O, sync for computation

Agent loops are mostly I/O — async is the default.

Log, and test the HTTP layer

logging over print. FastAPI plus httpx exercises the API without a live server.

Exercises

01 — Data processing: dataclasses, filtering, formatting

02 — Async pipeline: queues, timeouts, producer/consumer

03 — REST API: FastAPI CRUD with httpx tests



Module 0 — Complete

Next: working with the LLM